

STUDENT EXPENSE TRACKER

CSE1021 — Introduction to Problem Solving and Programming

Project Report

Prepared as a Python-based real-world problem-solving project

VIT Bhopal University

1. Introduction

The Student Expense Tracker is a menu-driven Python application designed to help students record and analyze their daily expenses. It applies fundamental programming and problem-solving concepts such as variables, input/output, conditional statements, loops, functions, lists, dictionaries, searching, counting, summation, and exception handling. The design deliberately remains simple so that each feature can be explained in terms of concepts covered by CSE1021.

2. Problem Statement

College students make many small expenses for food, transportation, shopping and other activities. Without a simple record, it can be difficult to determine total spending and category-wise spending. The proposed application provides a command-line solution for recording, viewing, searching, analyzing and deleting expenses.

3. Objectives

The objectives are to build a simple real-world Python application; demonstrate fundamental problem-solving concepts; use lists and dictionaries to represent expense data; implement searching, counting and summation; calculate total and average spending; provide category-wise analysis; and demonstrate input validation and error handling.

4. Functional Requirements

The application provides six menu choices: Add Expense, View Expenses, Search Expenses, Spending Summary, Delete Expense, and Exit. Add Expense accepts amount, category and description. View displays stored records. Search finds records by category. Spending Summary calculates total, count, average and category-wise spending. Delete removes a selected record. Input validation prevents invalid amounts, empty fields and invalid selections.

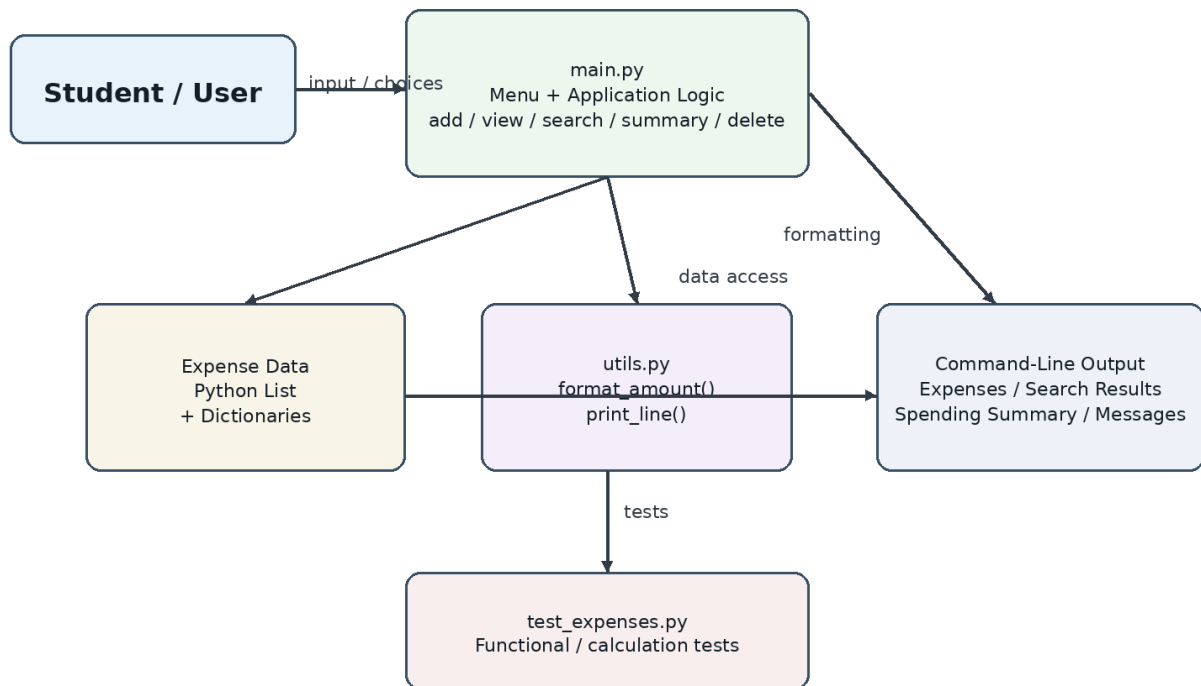
5. Non-Functional Requirements

The application is designed for usability through a simple menu-driven interface, reliability through input validation and exception handling, maintainability through separate functions, and resource efficiency through basic Python data structures without external runtime libraries.

6. System Architecture

The system consists of a user-facing command-line interface implemented in `main.py`, application functions for each operation, an in-memory list of dictionaries for expense data, utility functions in `utils.py`, and a separate `test_expenses.py` test module. The application does not use a database or network service.

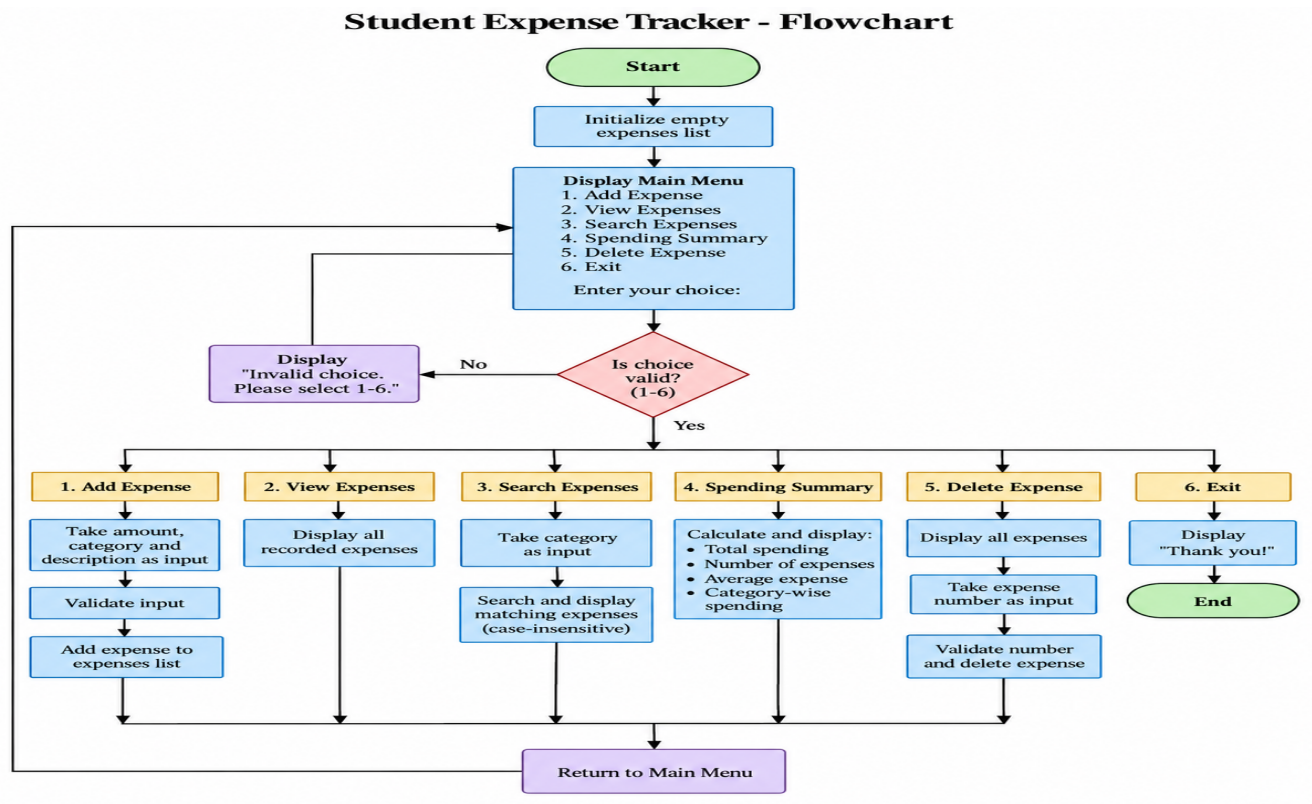
Student Expense Tracker — System Architecture



7. Design Diagrams

7.1 Workflow / Flowchart

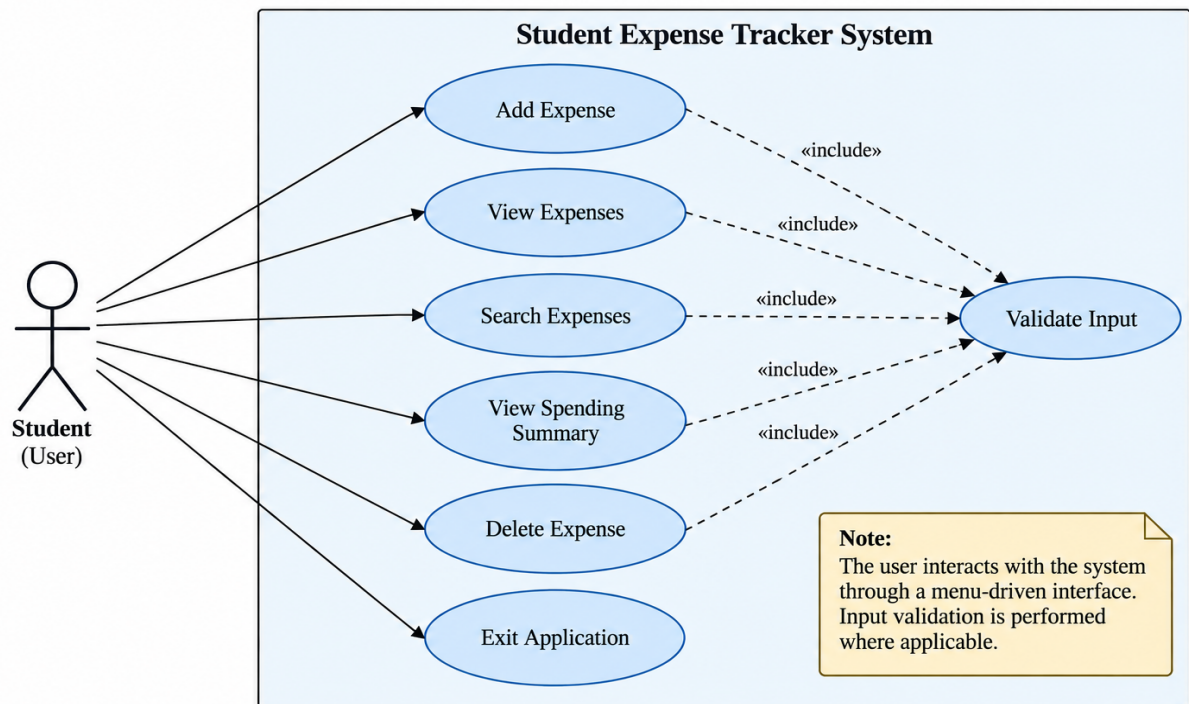
The workflow begins with initialization, displays the main menu, accepts a user choice, performs the selected operation, and returns to the menu until the user chooses Exit.



7.2 Use Case Diagram

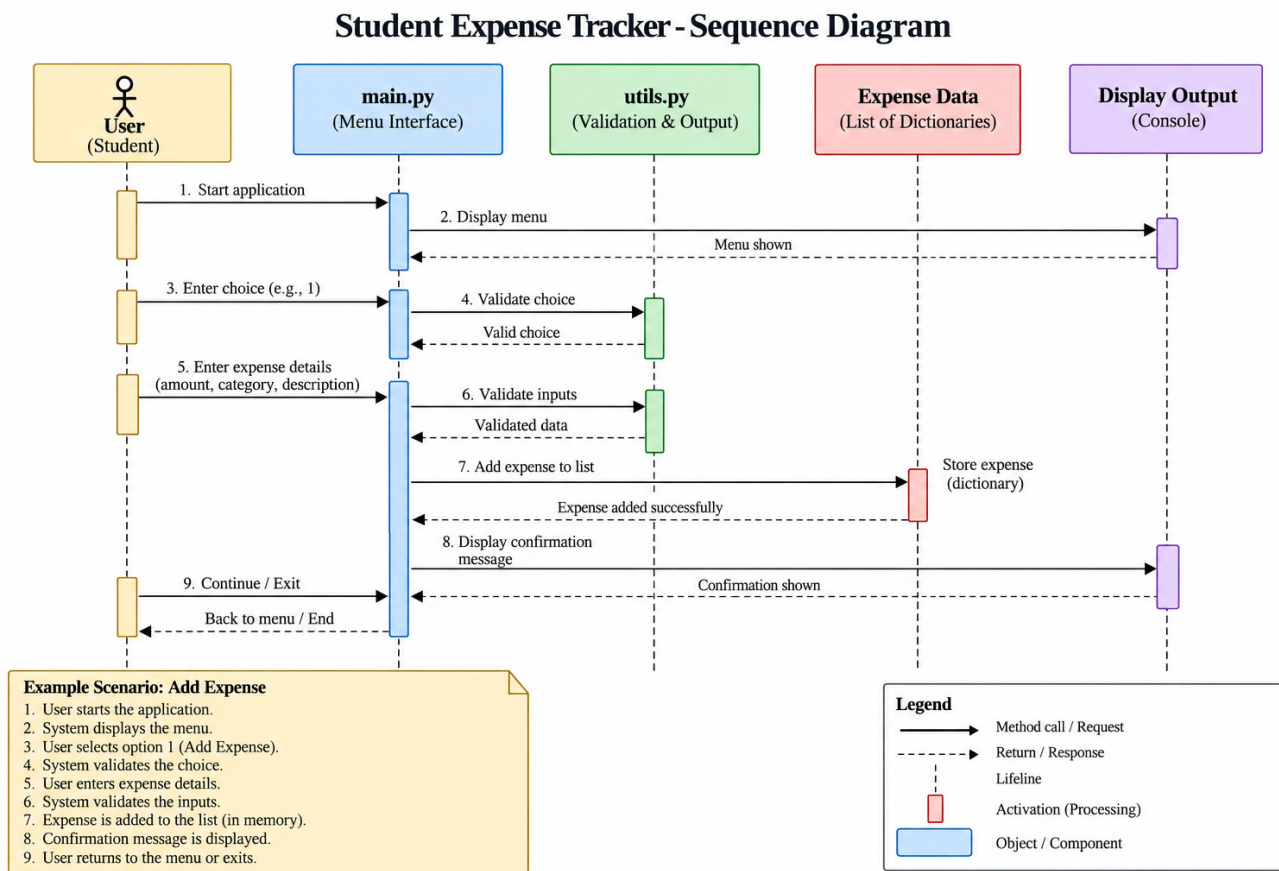
The Student/User interacts with the six main application actions. Input validation supports the operations that require user-entered data.

Student Expense Tracker - Use Case Diagram



7.3 Sequence Diagram

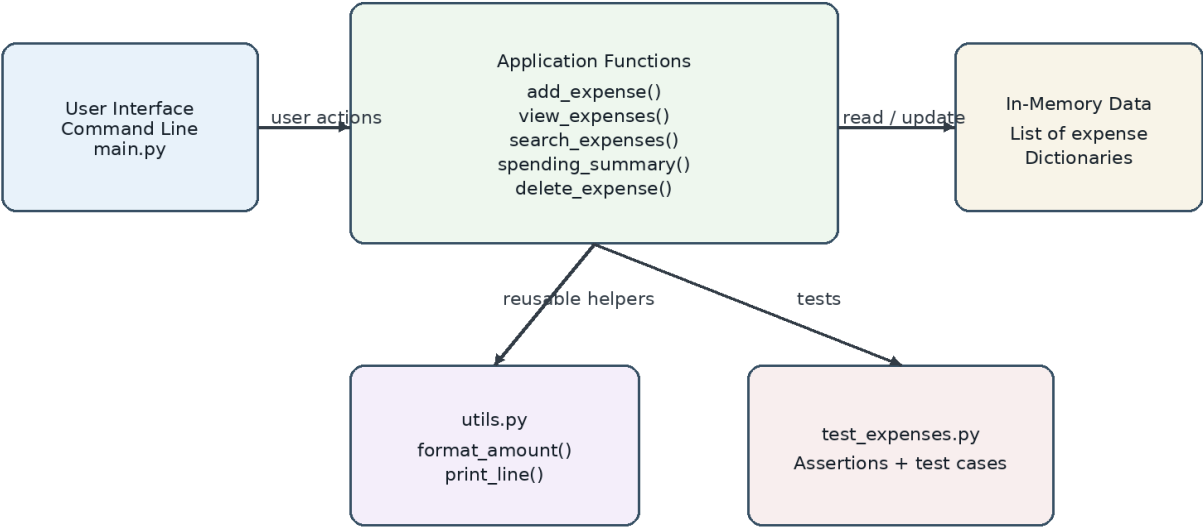
The sequence diagram illustrates an Add Expense interaction: the user selects the operation, enters expense information, the application validates the input, stores the record, displays confirmation, and returns to the menu.



7.4 Component Diagram

The component view matches the implemented modular structure: main.py contains the application functions, utils.py contains reusable helpers, the in-memory expense list stores records, and test_expenses.py performs basic checks.

Student Expense Tracker — Component Diagram



8. Data Representation

Each expense is represented by a Python dictionary containing amount, category and description. Multiple dictionaries are stored in a Python list. A second dictionary is created during spending-summary processing to group totals by category.

```
expense = {
    "amount": 120,
    "category": "Food",
    "description": "Dinner"
}

expenses = [expense]
```

9. Algorithms and Processing

Expense addition validates the amount and text fields before appending a dictionary to the list. Searching iterates through the list and performs a case-insensitive category comparison. Total spending is calculated by summing all expense amounts. Average spending divides the total by the number of expenses. Category-wise spending uses a dictionary to accumulate totals. Deletion uses the selected expense number to remove the corresponding list item.

10. Implementation Details

The implementation is divided into five primary functions: `add_expense()`, `view_expenses()`, `search_expenses()`, `spending_summary()`, and `delete_expense()`. The `main()` function controls the menu loop. `utils.py` provides `format_amount()` and `print_line()` so formatting logic is reusable.

11. Screenshots / Results

The following sample output demonstrates the spending-summary functionality using four example expenses. It is included as a representative program result.

STUDENT EXPENSE TRACKER – SAMPLE OUTPUT

```
=====
      STUDENT EXPENSE TRACKER
=====
1. Add Expense
2. View Expenses
3. Search Expenses
4. Spending Summary
5. Delete Expense
6. Exit
-----
Enter your choice: 4

===== SPENDING SUMMARY =====
Total Spending: INR 900.0
Number of Expenses: 4
Average Expense: INR 225.0

===== CATEGORY-WISE SPENDING =====
Food : INR 320.0
Transport : INR 80.0
Shopping : INR 500.0
```

12. Testing Approach

The project includes `test_expenses.py` with five test cases covering adding expense data, multiple expenses, category-wise calculation, total calculation and deletion. The current test execution completed successfully with all five tests passing.

```
=====
      STUDENT EXPENSE TESTS
=====

Test 1 passed: Add Expense
Test 2 passed: Multiple Expenses
Test 3 passed: Category Calculation
Test 4 passed: Total Calculation
Test 5 passed: Delete Expense

=====
      ALL TESTS PASSED!
=====
```

13. Challenges Faced

The main implementation challenges were organizing the menu-driven workflow, validating different kinds of user input, maintaining expense records using basic data structures, and separating reusable functionality into functions and a utility module while keeping the application simple.

14. Design Decisions and Rationale

A command-line interface was selected because the course focuses on fundamental programming and problem solving. Lists and dictionaries were selected because they are explicitly covered in the course and are sufficient for the current in-memory requirements. Functions were used to keep each operation modular. No database or external library was introduced because persistent storage is outside the current scope.

15. Learnings and Key Takeaways

The project demonstrates how a real-world problem can be broken into smaller computational tasks. Key learnings include designing a menu workflow, representing structured data with dictionaries, processing collections with loops, implementing searching and summation, handling invalid input, organizing code into functions, and writing basic automated tests.

16. Future Enhancements

Future versions could add file-based persistence, dates and times, monthly reports, budget limits, graphical charts, report export, a graphical interface, database integration, and user accounts.

17. Conclusion

The Student Expense Tracker provides a small but complete application of CSE1021 programming and problem-solving concepts. It solves a practical student problem while keeping the implementation understandable, modular and testable.

18. References

1. VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines.
2. CSE1021 — Introduction to Problem Solving and Programming, VIT Bhopal course syllabus.